

Analisi e comparazione dei sistemi DeFI che usano AMM e Liquidity Pools

Cocco Mattia 65336 e Lepuri Tomas 65358

Università degli Studi di Cagliari

Corso di Laurea Magistrale in Informatica

Blockchain and Smart Contracts

Prof. Andrea Pinna

Anno Accademico 2024-2025

© 2025 Mattia Cocco & Tomas Lepuri – Licenza CC BY 4.0

Indice

<i>Decentralized Finance</i>	4
<i>Liquidity pools</i>	5
Esempio implementativo	5
<i>Automated Market Makers</i>	8
Esempio implementativo	10
<i>Attori Principali</i>	12
Decentralized Exchanges	12
Liquidity providers e Yield Aggregators	13
Decentralized Autonomous Organizations.....	15
Oracoli, Wallet e API	16
Sicurezza	17
<i>Valutazione delle strategie di profitto</i>	19
Trading Fees.....	19
Liquidity Mining.....	20
Lending su LP Tokens.....	22
Servizi infrastrutturali	22
<i>Uniswap v4</i>	23
Hooks	23
Singleton e risparmio del gas	26
Funzioni elementari	27
<i>Scenario d'uso completo</i>	29
Creazione dei token	29
Automated Market Maker	30
Test	34
Vulnerabilità	38
Conclusioni	38
<i>Bibliografia</i>	39

Licenza	40
----------------------	-----------

Decentralized Finance

La DeFI, o **Finanza Decentralizzata**, consiste in un insieme di protocolli e applicazioni, generalmente basate su tecnologie blockchain, con lo scopo di offrire servizi finanziari senza la necessità di intermediari centralizzati, come banche, broker o assicurazioni, di cui bisogna necessariamente fidarsi.

Nella finanza tradizionale infatti (centralizzata), ogni transazione e servizio in generale sono gestiti da un ente centrale, rigidamente regolamentato da requisiti e autorizzazioni, con una trasparenza limitata e velocità delle operazioni strettamente legata ai ritmi lavorativi del settore.

Al contrario, la DeFI sfrutta gli **smart contracts**, programmi autonomi e immutabili, in continua esecuzione su blockchain pubbliche, come Ethereum, che permettono di automatizzare operazioni come scambi di valute, prestiti, gestione di assets e rendimenti, garantendo più accessibilità, trasparenza, rapidità e disponibilità dei servizi.

Possiamo infatti riassumere alcuni principi cardine:

- **Decentralizzazione o disintermediazione:** elimina l'esigenza di fiducia verso banche o altre istituzioni.
- **Trasparenza:** il codice è open-source e le transazioni sono registrate su blockchain pubblicamente consultabili.
- **Accessibilità:** è possibile accedere a questi servizi con meno controlli bancari o territoriali, dato che molti provider di servizi hanno comunque bisogno di dotarsi di misure di controllo e KYC per poter essere regolamentati nell'UE o simili.
- **Composability:** i protocolli DeFI possono essere integrati tra loro, creando applicazioni più complesse.

La nascita della Finanza Decentralizzata può essere ricondotta alla diffusione di Ethereum, che ha introdotto la possibilità di scrivere questi contratti intelligenti. Tra i primi protocolli, vale la pena citarne alcuni:

- **MakerDAO:** il protocollo che permise la prima stablecoin decentralizzata.
- **Compound:** un protocollo di tasso di interesse autonomo.
- **Uniswap:** uno swap di criptovalute decentralizzato, che verrà approfondito in seguito.

Con la sua evoluzione, questo ecosistema è arrivato a comprendere decine di miliardi di dollari in Total Value Locked.

Liquidity pools

Nell'ambito della finanza decentralizzata, un **Liquidity Pool** consiste in una riserva di due o più token che vengono bloccati tramite uno smart contract che consente agli utenti di scambiarli automaticamente, secondo regole predefinite, solitamente tramite un Automated Market Maker: un algoritmo che permette di scambiare token senza la necessità di un order book, come verrà approfondito in seguito.

Gli utenti che depositano token nel pool, formalmente stanno fornendo fondi a quest'ultimo e sono detti **Liquidity Providers** che ricevono in cambio degli LP tokens, rappresentanti della loro quota nel pool. In questo modo con l'aumentare del valore del pool tramite le fees, che la maggior parte degli AMM trattiene, i providers vedranno una plusvalenza della loro quota.

Naturalmente se il valore dei token (come ETH) diminuisce fortemente rispetto al momento del deposito, un Liquidity Provider può perdere denaro anche se il valore totale del pool è cresciuto tramite le fees, rischio chiamato **Impermanent Loss**.

Nello specifico la impermanent loss si verifica quando il prezzo relativo dei token di un pool cambia nel mercato esterno, quindi le proporzioni dei token nel pool cambiano secondo le regole specifiche di un AMM e i liquidity providers potrebbero trovarsi con un valore complessivo della quota inferiore a quello che avrebbero avuto i loro token se non li avessero depositati nel pool. Questa perdita è detta “non permanente” perché viene realizzata soltanto se la liquidità viene prelevata dal pool.

Esempio implementativo

Nel core contract di Uniswap v2¹ sono presenti alcune funzioni che permettono di comprendere meglio i concetti teorici.

La prima funzione restituisce le riserve correnti dei token e il timestamp dell'ultima operazione, utile per calcolare i prezzi medi ponderati nel tempo (TWAP), le riserve (*righe 39 e 40*) sono variabili di stato che vengono aggiornate a ogni swap o cambiamento della liquidità e servono a calcolare le quantità in entrata e in uscita:

```
38     function getReserves() public view returns (uint112 _reserve0, uint112 _reserve1, uint32 _blockTimestampLast) {
39         _reserve0 = reserve0;
40         _reserve1 = reserve1;
41         _blockTimestampLast = blockTimestampLast;
42     }
```

¹ Disponibile al link: <https://github.com/Uniswap/v2-core/blob/master/contracts/UniswapV2Pair.sol>.

La funzione **mint** invece viene chiamata ogni volta che un Liquidity Provider aggiunge liquidità al pool. Innanzitutto calcola il saldo aggiuntivo dei due token implementati con ERC20 e lo salva nelle variabili **amount0** e **amount1**, leggendo i bilanci dell'indirizzo del contratto alle *righe 112 e 113* e la differenza rispetto allo stato precedente del pool alle *righe 114 e 115*:

```
110     function mint(address to) external lock returns (uint liquidity) {
111         (uint112 _reserve0, uint112 _reserve1,) = getReserves(); // gas savings
112         uint balance0 = IERC20(token0).balanceOf(address(this));
113         uint balance1 = IERC20(token1).balanceOf(address(this));
114         uint amount0 = balance0.sub(_reserve0);
115         uint amount1 = balance1.sub(_reserve1);
```

In seguito, se un provider è il primo in assoluto, la liquidità viene calcolata come radice quadrata del prodotto delle quantità di token meno la quantità minima di liquidità, che viene bloccata permanentemente, come si può vedere alle *righe 120 e 121*, altrimenti è calcolata sulla base dei token aggiunti alle riserve esistenti, calcolando le proporzioni dei token nel pool, in modo da mantenerlo bilanciato (*riga 123*).

```
119         if (_totalSupply == 0) {
120             liquidity = Math.sqrt(amount0.mul(amount1)).sub(MINIMUM_LIQUIDITY);
121             _mint(address(0), MINIMUM_LIQUIDITY); // permanently lock the first MINIMUM_LIQUIDITY tokens
122         } else {
123             liquidity = Math.min(amount0.mul(_totalSupply) / _reserve0, amount1.mul(_totalSupply) / _reserve1);
124         }
```

Infine, se la liquidità è sufficiente, vengono emessi gli LP token nella *riga 126*, vengono aggiornate le riserve e viene emesso l'evento di **Mint**, come si nota alla *riga 130*:

```
125         require(liquidity > 0, 'UniswapV2: INSUFFICIENT_LIQUIDITY_MINTED');
126         _mint(to, liquidity);
127
128         _update(balance0, balance1, _reserve0, _reserve1);
129         if (feeOn) kLast = uint(reserve0).mul(reserve1); // reserve0 and reserve1 are up-to-date
130         emit Mint(msg.sender, amount0, amount1);
```

La funzione **burn**, al contrario, permette ai Liquidity Providers di prelevare la loro quota, “bruciando” gli LP token. Per prima cosa legge le riserve e i saldi attuali come nella mint, poi calcola quanti token vuole bruciare l’utente e quanti ne deve restituire all’indirizzo to (*righe 145 e 146*):

```
134     function burn(address to) external lock returns (uint amount0, uint amount1) {
135         (uint112 _reserve0, uint112 _reserve1,) = getReserves(); // gas savings
136         address _token0 = token0;                                // gas savings
137         address _token1 = token1;                                // gas savings
138         uint balance0 = IERC20(_token0).balanceOf(address(this));
139         uint balance1 = IERC20(_token1).balanceOf(address(this));
140         uint liquidity = balanceOf[address(this)];
141
142         bool fee0n = _mintFee(_reserve0, _reserve1);
143         uint _totalSupply = totalSupply; // gas savings, must be defined here since totalSupply can update in _mintFee
144         amount0 = liquidity.mul(balance0) / _totalSupply; // using balances ensures pro-rata distribution
145         amount1 = liquidity.mul(balance1) / _totalSupply; // using balances ensures pro-rata distribution
146         require(amount0 > 0 && amount1 > 0, 'UniswapV2: INSUFFICIENT_LIQUIDITY_BURNED');
```

In seguito, distrugge i token, aggiorna le riserve e genera l’evento **Burn** alla *riga 155*:

```
147         _burn(address(this), liquidity);
148         _safeTransfer(_token0, to, amount0);
149         _safeTransfer(_token1, to, amount1);
150         balance0 = IERC20(_token0).balanceOf(address(this));
151         balance1 = IERC20(_token1).balanceOf(address(this));
152
153         _update(balance0, balance1, _reserve0, _reserve1);
154         if (fee0n) kLast = uint(reserve0).mul(reserve1); // reserve0 and reserve1 are up-to-date
155         emit Burn(msg.sender, amount0, amount1, to);
```

Automated Market Makers

Un AMM, o **Automated Market Maker**, è un tipo di algoritmo, impiegato principalmente negli Exchange Decentralizzati, che consente lo scambio di token senza la necessità di una controparte diretta umana (senza un order book). Nei mercati tradizionali infatti, la domanda di beni o valute viene abbinata all'offerta, mentre gli AMM utilizzano una formula matematica per determinare il prezzo dei token, sulla base della disponibilità nei liquidity pools.

Gli exchange decentralizzati sono solitamente anche **non-custodial**, poiché gli utenti mantengono il completo controllo dei propri fondi e delle proprie chiavi private, dato che gli scambi avvengono on-chain, senza bisogno di depositare gli assets presso un'entità terza e i fondi rimangono nel wallet fino al momento dello swap.

Un noto esempio è Uniswap v2, che utilizza la seguente formula:

$$x \cdot y = k$$

Dove:

- x e y sono le quantità dei due token nel pool.
- k è una costante fissa.
- Ogni swap modifica x e y , mantenendo k costante.

Supponiamo di avere un pool ETH/DAI con:

- $x = 10$ ETH
- $y = 200$ DAI
- Quindi $k = 10 \cdot 200 = 2000$

Se uno swap aggiungesse 1 ETH al pool, l'utente riceverebbe meno DAI in cambio di ETH dato che dovrebbe essere soddisfatta $11 \cdot y = 2000$.

Questo è solo un modello di AMM, chiamato **Constant Product**, dato che k è una costante, il prezzo si adatta dinamicamente al rapporto tra le quantità dei token e la curva dei prezzi è iperbolica².

Esistono altri modelli di AMM, tra cui:

- **Constant Mean**: che generalizza il modello a più di due token, permette di dare pesi diversi ai token che influiscono sulla determinazione dei prezzi, con formula:

$$\prod_{i=1}^n x_i^{w_i} = k$$

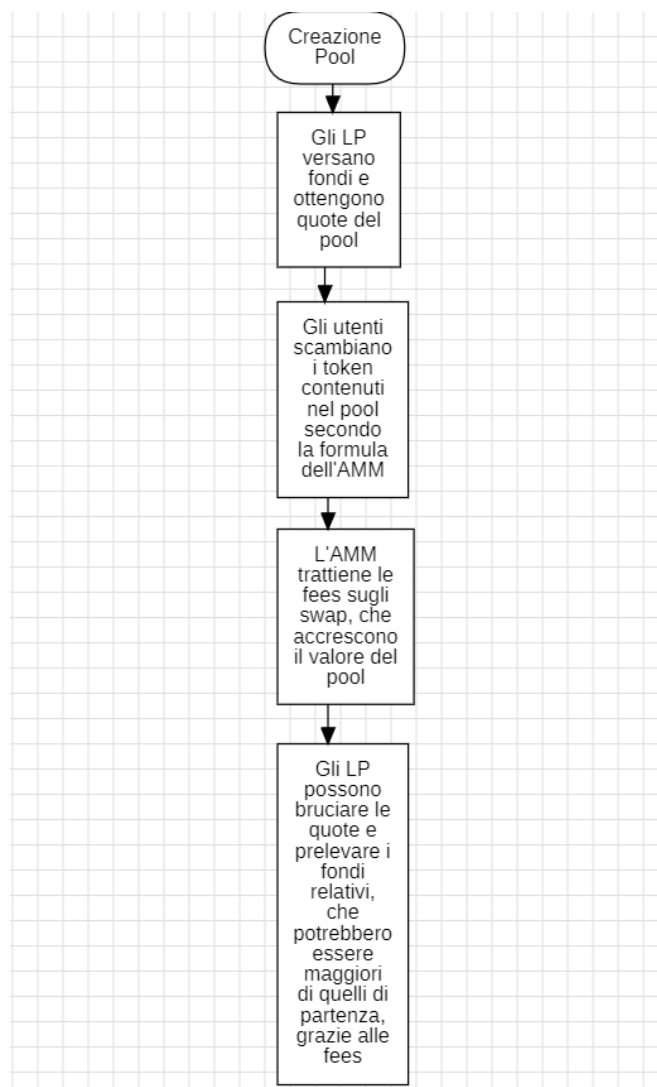
² La relazione $x \cdot y = k$ definisce un ramo di iperbole nel primo quadrante. Ogni scambio sposta il pool lungo questa curva e scambi maggiori implicano variazioni di prezzo maggiori, fenomeno noto come **slippage**.

- **Logarithmic Market Scoring Rule:** utilizzato per i prediction markets, permette di calcolare il prezzo marginale di ogni outcome³ e ha formula del costo:

$$C(q) = b \cdot \log \left(\sum_i e^{q_i/b} \right)$$

Dove q_i è la quantità acquistata dall'evento i e b è un parametro di liquidità.

Sostanzialmente, automated market makers e liquidity pools lavorano in simbiosi.



³ Il prezzo marginale di un outcome è la variazione del costo totale in funzione della variazione della quantità q_i acquistata dall'evento i : sostanzialmente la probabilità normalizzata di quell'evento.

Quindi:

- Il pool contiene i due token.
- Gli utenti possono swapparli.
- I Liquidity Providers guadagnano sulle commissioni.
- I prezzi vengono determinati dinamicamente sulla base del rapporto tra token.

Altri esempi di attori rilevanti nel campo degli AMM, oltre Uniswap, sono:

- **SushiSwap**: una fork di Uniswap che implementa delle differenze nel meccanismo di incentivazione per i fornitori di liquidità.
- **PancakeSwap**: AMM Constant Product che opera sulla Binance Smart Chain, con focus su token BEP-20.
- **Balancer**: AMM Constant Mean che consente la creazione di pool con più di due token e pesi personalizzabili associati.
- **Augur**: piattaforma di predizione del mercato che utilizza appunto un AMM LMSR.

Esempio implementativo

Anche in questo caso l'applicazione dei concetti appena descritti può essere osservata perfettamente nel **core smart contract** di Uniswap v2.

La funzione di **swap** permette agli utenti di scambiare un token per un altro, secondo la formula precedentemente descritta:

- L'utente dice quanto vorrebbe ricevere.
- Il contratto calcola se sono stati ricevuti abbastanza token in cambio, tenendo conto anche della fee con la formula matematica.
- In caso sia tutto corretto invia i token richiesti, aggiornando le riserve, altrimenti la transazione fallisce.

I parametri includono le quantità dei token e l'indirizzo che riceverà i token in uscita, in seguito controlla subito che almeno uno dei due amount sia positivo alla *riga 160* e che la liquidità del pool sia sufficiente alla *riga 162*:

```
159     function swap(uint amount0Out, uint amount1Out, address to, bytes calldata data) external lock {
160         require(amount0Out > 0 || amount1Out > 0, 'UniswapV2: INSUFFICIENT_OUTPUT_AMOUNT');
161         (uint112 _reserve0, uint112 _reserve1,) = getReserves(); // gas savings
162         require(amount0Out < _reserve0 && amount1Out < _reserve1, 'UniswapV2: INSUFFICIENT_LIQUIDITY');
```

Il contratto invia all'utente i token richiesti⁴ alle *righe 170 e 171* e verifica quanto è stato depositato in cambio, confrontando i saldi attuali con le riserve, per poi calcolare gli importi in ingresso alle *righe 176 e 177*. La funzione di swap gestisce i balance dei due token ERC20 tramite le variabili calcolate alle *righe 173 e 174*:

```
170         if (amount0Out > 0) _safeTransfer(_token0, to, amount0Out); // optimistically transfer tokens
171         if (amount1Out > 0) _safeTransfer(_token1, to, amount1Out); // optimistically transfer tokens
172         if (data.length > 0) IUniswapV2Callee(to).uniswapV2Call(msg.sender, amount0Out, amount1Out, data);
173         balance0 = IERC20(_token0).balanceOf(address(this));
174         balance1 = IERC20(_token1).balanceOf(address(this));
175     }
176     uint amount0In = balance0 > _reserve0 - amount0Out ? balance0 - (_reserve0 - amount0Out) : 0;
177     uint amount1In = balance1 > _reserve1 - amount1Out ? balance1 - (_reserve1 - amount1Out) : 0;
```

Viene applicata la **fee** di Uniswap dello 0.3%, che equivale a moltiplicare per 997/1000:

```
180         uint balance0Adjusted = balance0.mul(1000).sub(amount0In.mul(3));
181         uint balance1Adjusted = balance1.mul(1000).sub(amount1In.mul(3));
```

Viene effettuato il controllo fondamentale, che si assicura che nessuno possa ottenere più di quanto dovrebbe e che il valore del pool non diminuisca:

```
182         require(balance0Adjusted.mul(balance1Adjusted) >= uint(_reserve0).mul(_reserve1).mul(1000**2), 'UniswapV2: K');
183     }
```

Che equivale a controllare che:

$$(x + \Delta x \cdot 0.997) \cdot (y - \Delta y) \geq k$$

Infine vengono aggiornate le riserve e viene emesso l'**evento** di Swap, che verrà salvato nel log della blockchain:

```
185         _update(balance0, balance1, _reserve0, _reserve1);
186         emit Swap(msg.sender, amount0In, amount1In, amount0Out, amount1Out, to);
```

⁴ I token vengono inviati “ottimisticamente” prima dei controlli, che nel caso di esito negativo faranno fallire la transazione.

Attori Principali

Decentralized Exchanges

Sicuramente gli **Exchange Decentralizzati**, o **DEX**, hanno un ruolo fondamentale quando si parla di Finanza Decentralizzata che utilizza AMM e Liquidity Pools, dato che sono le piattaforme che effettivamente consentono lo scambio dei token tra utenti, senza intermediari.

Gli exchange tradizionali (centralizzati) custodiscono i fondi e gestiscono l'order book, mentre i DEX, tramite le tecnologie discusse nei capitoli precedenti, delegano tutta la logica di scambio a Smart Contracts su blockchain pubbliche.

I DEX più diffusi fanno uso di AMM per determinare i prezzi dinamicamente e facilitare le transazioni tramite Liquidity Pools.

Nello specifico, questo tipo di exchange svolge dei compiti precisi tra cui:

- **Gestione dei liquidity pools**

I DEX ospitano uno o più smart contracts che rappresentano dei pool di liquidità, formati da coppie di token.

Come si è già discusso, gli utenti possono ottenere una quota del pool, rappresentata dagli LP token, depositando entrambi i token nella proporzione richiesta.

- **Calcolo dei prezzi**

I prezzi, come anticipato, sono determinati tramite formule matematiche attraverso gli AMM, e non sulla base di domanda e offerta.

Ogni operazione di swap sposta l'equilibrio del pool, modificando i prezzi in tempo reale.

- **Permesso dello swap**

Gli utenti possono versare un token in cambio dell'altro, in cambio di una fee.

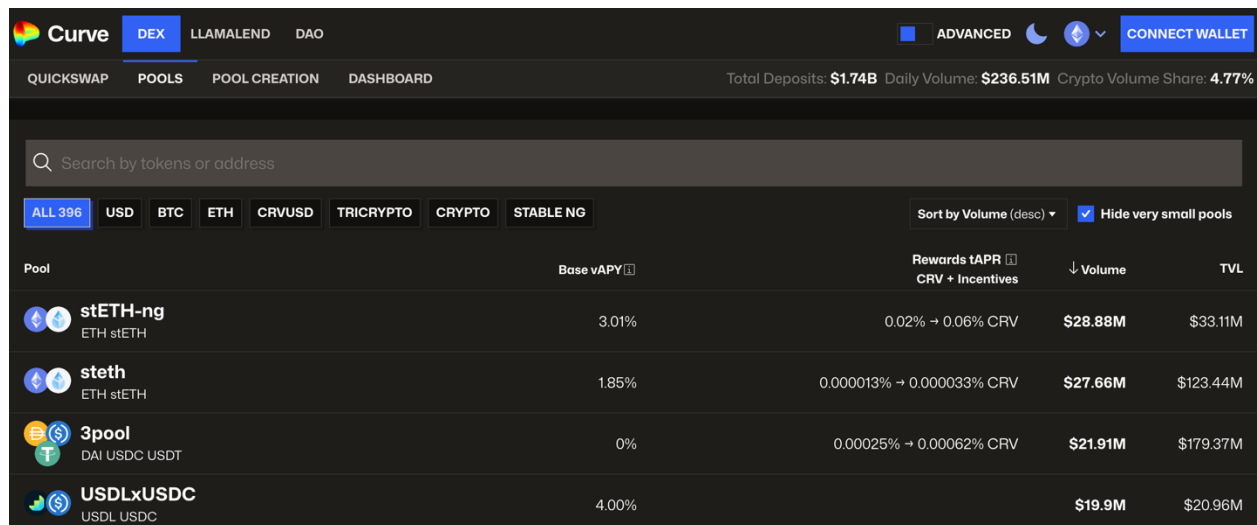
Alcune funzioni gestiscono gli output di questi swap, in base alle riserve e alla logica dell'AMM.

- **Distribuzione delle fees**

Le fees accumulate con gli scambi accrescono il valore del pool e vengono distribuite proporzionalmente ai liquidity providers, nel caso in cui decidano di ritirare la liquidità.

I maggiori esponenti sul mercato implementano principalmente i protocolli citati in termini di AMM:

- **Uniswap**
- **Balancer**
- **SushiSwap**
- **PancakeSwap**
- **Curve Finance**: DEX ottimizzato per gli stable coin.



The screenshot shows the Curve Finance DEX interface. At the top, there's a navigation bar with 'Curve' logo, 'DEX' tab, and links to 'LLAMALEND' and 'DAO'. A status bar shows 'Total Deposits: \$1.74B', 'Daily Volume: \$236.51M', and 'Crypto Volume Share: 4.77%'. Below is a search bar and filter tabs for 'ALL 396', 'USD', 'BTC', 'ETH', 'CRVUSD', 'TRICRYPTO', 'CRYPTO', and 'STABLE NG'. The main table lists pools with columns for 'Pool', 'Base vAPY', 'Rewards tAPR', 'Volume', and 'TVL'.

Pool	Base vAPY	Rewards tAPR	Volume	TVL
stETH-ng ETH stETH	3.01%	0.02% → 0.06% CRV	\$28.88M	\$33.11M
steth ETH stETH	1.85%	0.000013% → 0.000033% CRV	\$27.66M	\$123.44M
3pool DAI USDC USDT	0%	0.00025% → 0.00062% CRV	\$21.91M	\$179.37M
USDLxUSDC USDL USDC	4.00%		\$19.9M	\$20.96M

Liquidity providers e Yield Aggregators

È stato già spiegato come la figura del liquidity provider abbia il ruolo di depositare i fondi in cambio di LP token, permettendo agli utenti di scambiare i token e ottenendo una quota che potrebbe aumentare di valore grazie alle fees.

Nel settore della DeFi, tuttavia, la maggior parte delle realtà si serve principalmente di liquidity providers istituzionali o aziendali, come hedge funds o società proprietarie in grado di versare quantità significative di capitali e gestirli attivamente, dato che la liquidità in larga scala riduce lo **slippage** e aumenta la profondità del mercato, rendendo i DEX più attraenti per gli utenti finali.

Lo slippage, come accennato, è la differenza tra il prezzo atteso e il prezzo effettivo di esecuzione on-chain, che può verificarsi specialmente quando la dimensione di un'operazione non è trascurabile rispetto a quella del pool.

Molti DEX fungono anche da Liquidity Providers per le loro piattaforme, tuttavia esistono altre aziende specializzate nell'offrire liquidità per servizi finanziari di questo tipo, tra cui **Galaxy Digital**, **GSR Markets**, **Cumberland** e **Jump Crypto**.

Oltre al principale compito di fornire liquidità, per il funzionamento del sistema, le aziende che agiscono da liquidity providers possono avere anche altri ruoli:

- **Servizi da broker e custodia:**

Molti LP offrono accesso alla liquidità per effettuare operazioni a margine e infrastrutture per la custodia dei token, con la possibilità di prestiti. Per questo motivo c'è bisogno di sistemi robusti che implementino multi-sig, audit avanzati e siano orientati alla gestione del rischio per operare a livello istituzionale.

Tra questi vale la pena citare **Fireblocks** e **Alloy Capital**.

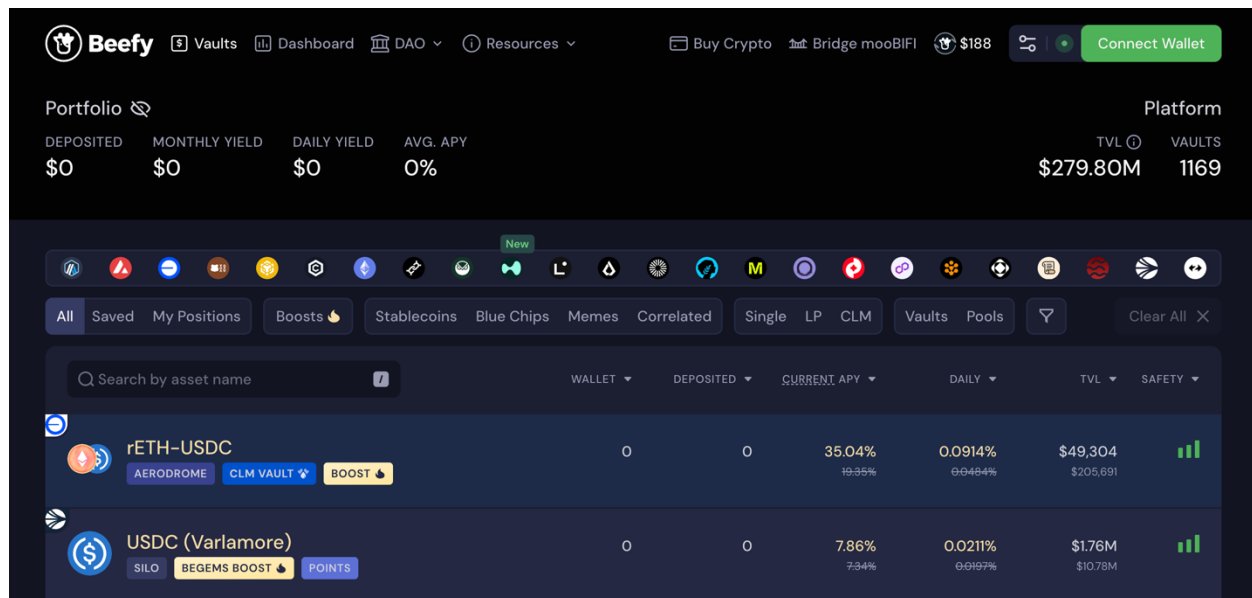
- **Ottimizzazione della liquidità:**

I liquidity providers cercano di massimizzare i ricavi dalle fees e minimizzare il rischio di impermanent loss con alcune tecniche.

Per esempio si possono adottare strategie di **distribuzione della liquidità** su più pool e protocolli, tenendo conto di prezzi, volatilità e volumi di trading, in modo da minimizzare il rischio e cercare di massimizzare i guadagni.

Un'altra tecnica molto utilizzata è la **Yield Aggregation**, che prevede l'automatizzazione della raccolta dei rendimenti derivati da più protocolli e del loro successivo reinvestimento secondo strategie specifiche.

Yearn Finance e **Beefy** sono tra i principali Yield Optimizer.



Inoltre è importante specificare che le istituzioni tendono a scegliere pool con assets stabili o pool protetti, con meno volatilità ma rendimenti più contenuti, in modo da ridurre il rischio di perdita.

Decentralized Autonomous Organizations

Per gestire in maniera comunitaria l'evoluzione dei protocolli e del progetto generale, i DEX fanno uso dei corrispettivi DAO, come:

- **Uniswap DAO.**
- **Curve DAO.**
- **Balancer DAO.**

Tramite i DAO, è possibile gestire tutte le decisioni in maniera decentralizzata e automatizzata tramite smart contracts.

Alcuni DAO hanno il proprio token per consentire le votazioni, che principalmente riguardano i seguenti argomenti:

- **Definizione parametri:**
Decisioni riguardo aggiornamenti di fees, incentivi e smart contracts.
- **Gestione treasury:**
Impegno nella crescita sostenibile e cambiamenti riguardanti finanziamenti, marketing e sviluppo.
- **Situazioni critiche:**
Alcuni DAO, come l'**Emergency DAO** di Curve incrementano la sicurezza con framework di emergenza.
- **Distribuzione potere politico:**
Il bilanciamento tra poteri è gestito con modelli come **veCRV** che supportano il voto token-based.

Oracoli, Wallet e API

Sono importanti anche tutte quelle infrastrutture che offrono strumenti per dati esterni, portafogli e interfacce utente.

Sicuramente hanno un ruolo fondamentale:

- **Oracoli:**

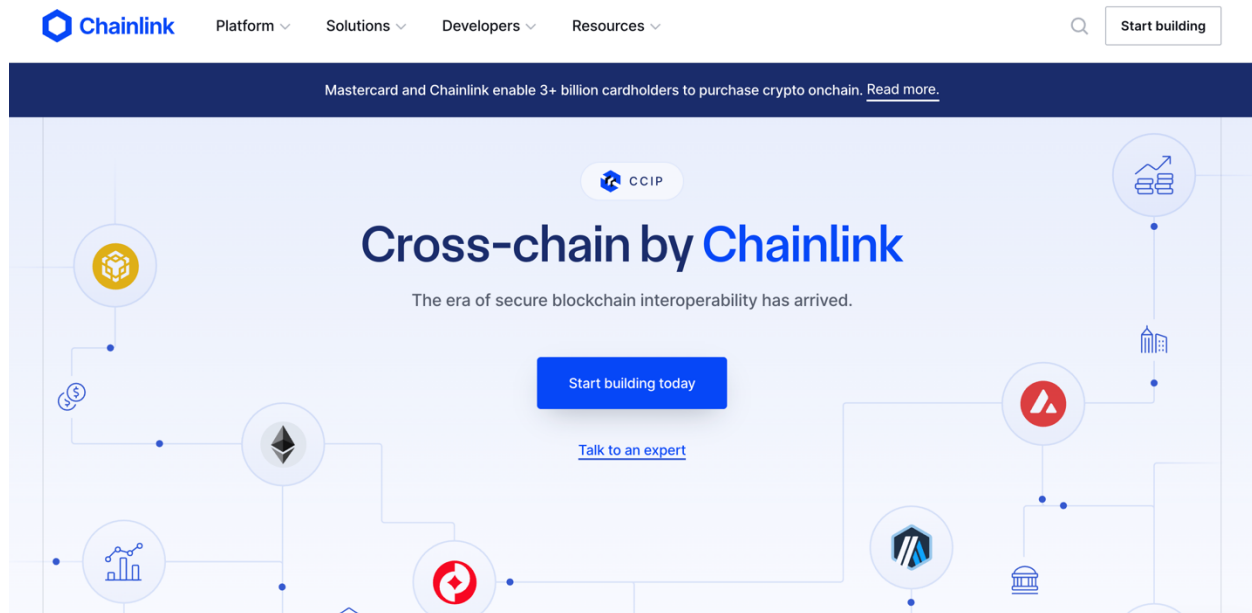
Sono importanti per la fornitura di prezzi esterni, in modo da evitare possibili manipolazioni. Tra queste si può citare **ChainLink**, che connette istituzioni finanziarie e blockchain da questo punto di vista.

- **Wallets:**

Offrono in genere interfacce per gestire i fondi e i token di cui si è in possesso, a volte possono essere utilizzati anche per gestire gli LP token e le quote di possesso dei pool. Sicuramente il più noto resta **Metamask**.

- **API:**

Permettono la connessione con le librerie Web3 e l'interazione con la blockchain.



Sicurezza

Infine, nel campo della DeFI, dove un errore può causare perdite di milioni di dollari, è fondamentale verificare e proteggere il codice, data la sua natura non intermediata.

Per questo motivo, alcune aziende si sono specializzate nella revisione completa del codice degli smart contracts e vengono ingaggiate dai DEX prima e durante il deploy.

Queste aziende ricoprono diversi compiti e rientrano in diverse categorie:

- **Smart Contract Audits:**

Gli audits sono revisioni approfondite e in genere indipendenti del codice, che includono verifica della conformità agli standard, analisi automatiche e manuali, simulazione di exploit, analisi formale della sicurezza e report dettagliati a riguardo.

Sicuramente tra i giganti del settore si può nominare **Openzeppelin**.

- **Bug Bounty:**

Molti protocolli di DeFI lavorano affiancandosi a delle piattaforme che offrono ricompense monetarie in cambio di segnalazioni di bug o falle di sicurezza, in genere stilando una classifica della gravità delle falle.

Questi programmi solitamente incentivano ricercatori e “white-hat hackers” a trovare le vulnerabilità che sono sfuggite agli audits, in modo da ridurre il tempo di esposizione delle vulnerabilità reali.

Alcuni protocolli, tra cui Curve, hanno Bug Bounties su piattaforme come **Immunefi**, dove la ricompensa per la rilevazione delle falle più gravi possono raggiungere anche centinaia di migliaia di dollari.

- **Strumenti di sicurezza:**

Altri tool garantiscono il monitoraggio, il deployment sicuro e l'automazione difensiva.

Alcuni sistemi di **real-time monitoring** permettono l'osservazione del comportamento on-chain dei protocolli, tracciando eventi come swap sospetti, picchi di slippage o cambiamenti anomali di liquidità, consentendo allerte immediate via webhooks o dashboards (come **Tenderly**) e pause di emergenza o interventi di governance per intervenire tempestivamente sui problemi.

Altre piattaforme si occupano di simulare grandi volumi su testnet prima di effettuare il deploy di un protocollo in mainnet e sono specializzate nell'utilizzo di script di deploy verificati, per evitare errori di configurazione e nella protezione dei deployer e degli admin con multisig e altre tecniche.

Infine esistono anche degli strumenti automatizzati che intervengono in caso di anomalie, come i **circuit breakers** che sospendono un contratto in caso di condizioni critiche.



Platform ▾

Security Researchers ▾

Blog

Login

Explore bounties



Join the **Immunefi All Stars**: the world's best hacker program

[Learn more](#)

Immunefi Magnus

One Platform. Unified Security Operations. Complete Onchain Protection.

[Get Started](#)

Valutazione delle strategie di profitto

Gli utenti finali adoperando strumenti di DeFi in alcuni casi possono ottenere profitto, per esempio, tramite Staking, Trading e Arbitraggio (spesso illecito), tuttavia queste sono operazioni che espongono l'utente a un certo rischio, spesso in rapporto ad un guadagno minimo.

In questo settore, il vero profitto viene fatto dalle aziende che gestiscono l'infrastruttura, in seguito analizzeremo le principali strategie di profitto adoperate nel settore degli AMM.

Trading Fees

Come è già stato accennato più volte, la fonte primaria di guadagno sono le fees guadagnate dagli scambi dei token, per questo motivo la maggior parte delle aziende che gestiscono DEX operano anche come Liquidity Providers.

Ogni volta che un utente intende scambiare un token per un altro, l'AMM calcola una fee sulla base della quantità di token scambiato e l'utente riceve la quantità di token che gli spetta secondo la formula specifica dell'AMM, meno questa fee che sta pagando per il servizio di swap.

Quindi, ogni scambio nel pool comporta una commissione che viene raccolta nel pool e redistribuita ai Liquidity Provider se ritirano i loro fondi. A volte parte di questa fee viene trattenuta dal protocollo o dalla tesoreria dell'azienda.

Nella maggior parte dei casi le fee sono inferiori allo 0.5%, dato che più le fee sono basse più un exchange è appetibile al pubblico, alcuni esempi sono:

- **Uniswap v2:** 0.3%
- **Uniswap v3/v4:** 0.01-0.4%
- **Curve:** 0.04%

Analizzando il report mensile di Uniswap, nel Giugno 2025 su un volume di **103.8 miliardi** di dollari di trading volume, sono state guadagnate **59.8 milioni** di dollari di fees, di cui 661.6 mila dollari sono stati trattenuti in tesoreria.

Financial statement	Jul 2025 Jul 1 - Jul 31	Jun 2025 Jun 1 - Jun 30
GMV		
Net deposits	\$7.5 B 6.2%	\$7.9 B 3.9%
Trading volume	\$1.9 B 98.2%	\$103.8 B 16.5%
Income statement		
Fees	\$1.2 M 98.0%	\$59.8 M 18.4%
(Supply-side fees)	\$1.2 M 98.0%	\$59.8 M 18.4%
Revenue	\$0.0 N/A ⓪	\$0.0 N/A ⓪
(Expenses)	\$277.7 K 96.7%	\$8.4 M 7.3%
(Cost of revenue)	\$11.8 K 96.5%	\$340.0 K 52.6%
(Token incentives)	\$265.9 K 96.7%	\$8.0 M 8.9%
Gross profit	\$-11.8 K 96.5%	\$-340.0 K 52.6%
Earnings	\$-277.7 K 96.7%	\$-8.4 M 7.3%
Treasury		
Treasury	\$2.5 B 5.6%	\$2.6 B 15.1%
Treasury (net)	\$12.2 K 98.2%	\$661.6 K 241.7%

Le fees corrispondono a circa lo 0,057% del volume di scambio, probabilmente dovuto al fatto che la commissione su Uniswap v3/v4 è configurabile a partire dallo 0.01%.

Liquidity Mining

Alcune aziende che lanciano exchange decentralizzati, piattaforme di prestiti e altri servizi di DeFi possono incentivare gli utenti a fornire liquidità al protocollo in cambio di token, come avviene per le ICO.

Questi token non sono regalati casualmente ma seguono un piano (tokenomics):

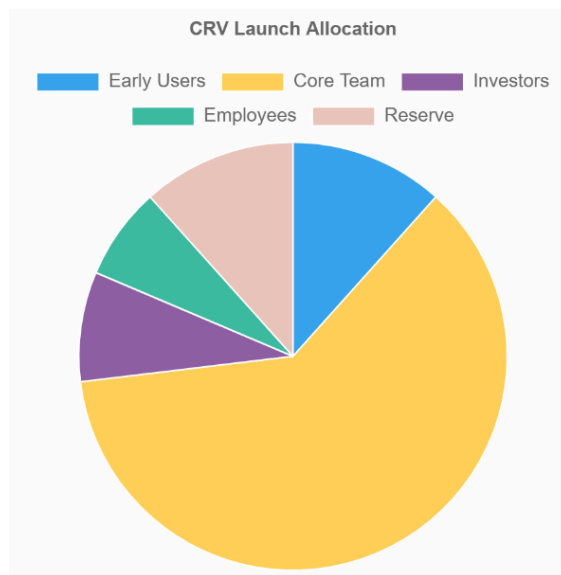
- Il team/protocollo trattiene una grossa porzione dei token.
- I token distribuiti fanno crescere il Total Value Locked.
- Vengono attirati più utenti che generano più fees.
- Inoltre il valore del token dovrebbe aumentare sul lungo termine, permettendo all'azienda di guadagnare anche attraverso la grossa parte di token trattenuta.

In sintesi, nel **liquidity mining** i token sono utilizzati come strumento di acquisizione utenti, che forniscono liquidità in cambio dei token, nella speranza che il token aumenti di valore e/o in cambio di privilegi riservati a chi li possiede, come il diritto di voto per quanto riguarda le decisioni relative al protocollo. Inoltre man mano che i token circolanti aumentano di numero, anche il volume di trading e le fees aumentano, creando una sorta di circolo vizioso che può persino far aumentare di valore il token e portare a guadagni futuri attraverso ecosistemi basati su staking, voti e reward.

Alcuni protocolli cercano di ottenere dei loop fornendo incentivi per mettere in staking e quindi bloccare delle quantità di token, incoraggiando l'utente a investire e ottenendo così più liquidità.

Un esempio esplicativo è CRV token, lanciato da Curve:

- I Liquidity Provider guadagnano dei CRV token.
- I CRV token possono essere rivenduti oppure possono essere bloccati (veCRV) per aumentare di 2.5 volte i futuri guadagni di CRV token, ottenere il permesso di votare nelle proposte della governance e guadagnare le fees sugli swaps.



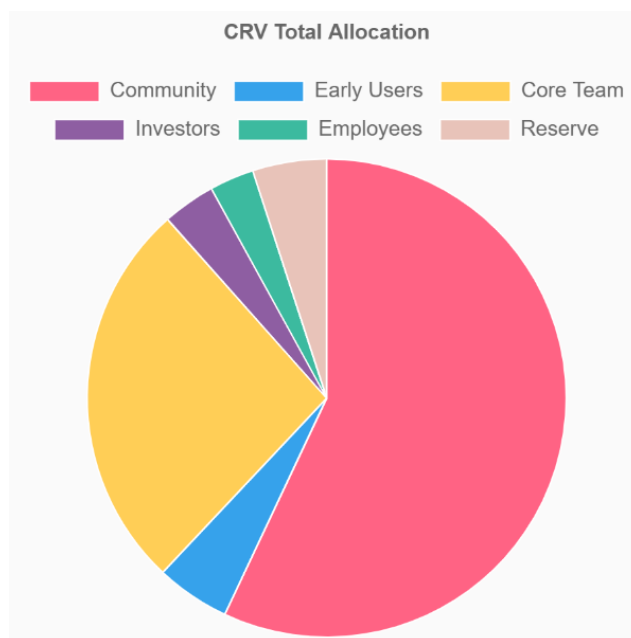
Al momento del lancio, 13 Agosto del 2020, il 61.47% dei token appartenevano appunto al Core Team e i token erano interamente bloccati in contratti chiamati Vesting Contracts predisposti per sbloccarli gradualmente a 1-4 anni dal lancio.

Il core team deteneva la maggior parte di token inizialmente, sperando di guadagnare in futuro anche da un eventuale aumento di valore di CRV. Inoltre dando la possibilità di guadagnare token in cambio di liquidità, Curve incentiva i liquidity providers ad aderire al protocollo, generando il loop positivo legato all'aumento di volume.

Tuttavia per adesso il valore del token CRV è sceso drasticamente dal momento del lancio.



Anche la distribuzione dei token è cambiata, dato che in seguito al lancio, i token appartenenti al core team si sono gradualmente sbloccati ma non sono aumentati di numero, mentre i liquidity providers hanno iniziato a guadagnare token e a bloccarli per usufruire dei privilegi sopracitati, andando così ad aumentare la quota comunitaria rispetto ai token totali emessi:



Lending su LP Tokens

Un altro metodo con cui le aziende riescono a generare profitto consiste nel massimizzare il rendimento di una posizione in un Liquidity Pool, prendendo prestiti in cambio di LP tokens usati come collaterali, per poi riciclare la liquidità attraverso strategie complesse.

Un esempio può essere quello di riciclare liquidità versandola in un protocollo in cambio di LP tokens, prendere stablecoins in prestito utilizzando gli LP tokens come collaterali e reinvestire anche gli stablecoins nel pool.

In questo modo le aziende e i Power Users possono ottenere un effetto leva, che comporta un aumento del profitto ma sicuramente espone a un rischio altrettanto maggiore.

Nella pratica, si può utilizzare Uniswap in combinazione con Aave per fornire liquidità ai pool e prendere prestiti con LP token come collaterali.

Servizi infrastrutturali

Infine, molte piattaforme riescono a generare ulteriori guadagni integrando strumenti come esecuzione automatica di funzioni, servizi di sicurezza o personalizzazioni, guadagnando attraverso fees per chiamate on-chain e abbonamenti premium.

Uniswap v4

Uniswap è uno dei leader nel campo degli Exchange Decentralizzati (non-custodial) basati su AMM.

La versione 4 è l'ultimo protocollo di casa Uniswap, lanciato in mainnet a Gennaio 2025, erede del v2 e del v3. In particolare si tratta di un protocollo di AMM con una peculiarità rispetto ai predecessori: permette la creazione di features personalizzate sul protocollo Uniswap, senza il bisogno di progettare e creare nuovi AMM da zero. Questa libertà di personalizzazione è resa possibile attraverso gli “**hooks**”, che permettono a sviluppatori e Liquidity Providers di eseguire azioni in momenti specifici della vita del pool.

Uniswap offre anche l'accesso a deep liquidity e altre features esistenti, aggiungendo la possibilità di livelli di fees illimitati, che permettono la personalizzazione dei pool e una maggiore efficienza dei costi.

I pool di liquidità vengono gestiti con un singolo smart contract, chiamato **singleton**, a scopo di ridurre drasticamente la spesa di gas nella creazione di un pool, che inoltre offre un sistema di accounting rapido, che consente solo il trasferimento dei saldi netti attraverso gli swap.

Uniswap v4 è retrocompatibile concettualmente con v3 ma presenta un'architettura molto diversa, in particolare:

- Il core del **pricing** può ancora utilizzare modelli constant product, ma è reso modulare.
- Mint, Burn e Swap esistono ancora, ma sono gestite attraverso un'interfaccia chiamata **PoolManager**.
- Ogni pool è gestito virtualmente dal **singleton** e non è un contratto autonomo con un suo storage per reserves, fees e liquidity come in v3.
- Per creare un **nuovo pool** non si crea un nuovo contratto ma si aggiungono nuove entry al singleton.
- **ETH nativo** è supportato nativamente, mentre nelle versioni precedenti veniva utilizzato WETH, come token ERC-20 per impacchettare ETH, necessario per le interfacce.
- La **custom logic** è possibile per ogni pool, integrabile tramite hooks.

Hooks

Gli hooks rappresentano una delle principali novità introdotte in Uniswap v4 e consistono in degli smart contracts che agiscono sostanzialmente come plugin per modificare il comportamento dei pool, degli swap, delle fees e della liquidità, un po' come le estensioni aggiungono funzionalità al browser.

Questa personalizzazione viene resa possibile dall'esecuzione di azioni in momenti specifici, come prima di uno swap o dopo l'aggiunta di liquidità.

All'atto pratico il contratto hook viene definito durante la creazione del pool e viene chiamato dal PoolManager in corrispondenza degli eventi chiave a cui è stato associato, per esempio:

beforeInitialize, afterSwap, beforeSwap, beforeAddLiquidity...

Un esempio semplice ed esplicativo di hook si può trovare nella repository di templates per creare hook (uniswapfoundation/v4-template).

L'hook **Counter** permette di contare quante volte vengono effettuate le operazioni di swap e di aggiunta e rimozione di liquidità su un pool. Per iniziare, il pool viene associato a 4 contatori e viene creato il costruttore che eredita da BaseHook alla *riga 27*:

```
13  contract Counter is BaseHook {
14      using PoolIdLibrary for PoolKey;
15
16      // NOTE: -----
17      // state variables should typically be unique to a pool
18      // a single hook contract should be able to service multiple pools
19      // -----
20
21      mapping(PoolId => uint256 count) public beforeSwapCount;
22      mapping(PoolId => uint256 count) public afterSwapCount;
23
24      mapping(PoolId => uint256 count) public beforeAddLiquidityCount;
25      mapping(PoolId => uint256 count) public beforeRemoveLiquidityCount;
26
27      constructor(IPoolManager _poolManager) BaseHook(_poolManager) {}
```

In seguito viene creata una funzione che restituisce una struttura di permessi, che specifica su quali eventi del pool vogliamo abilitare la callback; in questo caso solo **beforeSwap, AfterSwap, beforeAddLiquidity, beforeRemoveLiquidity**:

```
29  function getHookPermissions() public pure override returns (Hooks.Permissions memory) {
30      return Hooks.Permissions({
31          beforeInitialize: false,
32          afterInitialize: false,
33          beforeAddLiquidity: true,
34          afterAddLiquidity: false,
35          beforeRemoveLiquidity: true,
36          afterRemoveLiquidity: false,
37          beforeSwap: true,
38          afterSwap: true,
39          beforeDonate: false,
40          afterDonate: false,
41          beforeSwapReturnDelta: false,
42          afterSwapReturnDelta: false,
43          afterAddLiquidityReturnDelta: false,
44          afterRemoveLiquidityReturnDelta: false
45      });
46  }
```

Infine vengono definite le funzioni che verranno chiamate in corrispondenza degli eventi relativi al pool che, come si poteva intuire, incrementano i contatori, come si può notare alle righe 57 e 66:

```
52     function _beforeSwap(address, PoolKey calldata key, SwapParams calldata, bytes calldata)
53         internal
54         override
55         returns (bytes4, BeforeSwapDelta, uint24)
56     {
57         beforeSwapCount[key.toId()]++;
58         return (BaseHook.beforeSwap.selector, BeforeSwapDeltaLibrary.ZERO_DELTA, 0);
59     }
60
61     function _afterSwap(address, PoolKey calldata key, SwapParams calldata, BalanceDelta, bytes calldata)
62         internal
63         override
64         returns (bytes4, int128)
65     {
66         afterSwapCount[key.toId()]++;
67         return (BaseHook.afterSwap.selector, 0);
68     }
```

La personalizzabilità permessa tramite gli hooks è sicuramente un enorme miglioramento rispetto a Uniswap v3 e rappresenta una delle novità introdotte da questo protocollo, tuttavia questi componenti introducono anche alcune **vulnerabilità**, legate al fatto che il codice dell'hook rimane critico, nonostante i controlli eseguiti dal PoolManager sui permessi.

Un hook malevolo o mal implementato può condurre alla perdita di fondi o stalli del pool e per questo motivo nella documentazione di Uniswap si ricorda spesso di utilizzare solo gli hooks di cui ci si fida:

Note: Hooks are developed by independent third-party developers who are not affiliated with Uniswap Labs. Exercise caution when adding hooks, as some may be malicious or cause unintended consequences. See more about hooks [here](#).

Alcune aziende si stanno già specializzando sull'esecuzione di audit su questi smart contracts. Possiamo citare **Hacken** e **CertiK**.

Singleton e risparmio del gas

Come già anticipato, una differenza fondamentale rispetto alle versioni precedenti di Uniswap è il fatto che i pool vengano gestiti attraverso un unico contratto, detto singleton, che corrisponde al **PoolManager**.

Nel PoolManager si può osservare come tutti i pool vengano gestiti attraverso una singola mappa:

```
93      mapping(PoolId id => Pool.State) internal _pools;
```

Permettendo al singolo contratto di gestire tutte le logiche comuni, come swap, aggiunta e rimozione di liquidità ed evitando la duplicazione del codice relativo.

La funzione di **inizializzazione**, permette di creare un nuovo pool attraverso PoolManager:

- Vengono effettuati dei controlli per evitare **overflow**.
- Viene generato l'**id** univoco del pool.
- Le **variabili** del pool vengono salvate in un'unica mappa nel PoolManager basata sull'id.
- Viene emesso l'**evento** di inizializzazione.
- Viene restituito l'intervallo discreto del prezzo iniziale (**tick**).

Nella funzione **initialize** è anche possibile notare `beforeInitialize` e `afterInitialize` alle *righe 130 e 141*, che richiamerebbero l'attenzione di eventuali hook interessati a questi eventi:

```
117     function initialize(PoolKey memory key, uint160 sqrtPriceX96) external noDelegateCall returns (int24 tick) {
118         // see TickBitmap.sol for overflow conditions that can arise from tick spacing being too large
119         if (key.tickSpacing > MAX_TICK_SPACING) TickSpacingTooLarge.selector.revertWith(key.tickSpacing);
120         if (key.tickSpacing < MIN_TICK_SPACING) TickSpacingTooSmall.selector.revertWith(key.tickSpacing);
121         if (key.currency0 >= key.currency1) {
122             CurrenciesOutOfOrderOrEqual.selector.revertWith(
123                 Currency.unwrap(key.currency0), Currency.unwrap(key.currency1)
124             );
125         }
126         if (!key.hooks.isValidHookAddress(key.fee)) Hooks.HookAddressNotValid.selector.revertWith(address(key.hooks));
127
128         uint24 lpFee = key.fee.getInitialLPFee();
129
130         key.hooks.beforeInitialize(key, sqrtPriceX96);
131
132         PoolId id = key.toId();
133
134         tick = _pools[id].initialize(sqrtPriceX96, lpFee);
135
136         // event is emitted before the afterInitialize call to ensure events are always emitted in order
137         // emit all details of a pool key. poolkeys are not saved in storage and must always be provided by the caller
138         // the key's fee may be a static fee or a sentinel to denote a dynamic fee.
139         emit Initialize(id, key.currency0, key.currency1, key.fee, key.tickSpacing, key.hooks, sqrtPriceX96, tick);
140
141         key.hooks.afterInitialize(key, sqrtPriceX96, tick);
142     }
```

La creazione di un pool come dati aggiuntivi di un contratto già esistente senza dubbio riduce di molto la spesa di gas legata al deploy, dato che nelle versioni precedenti per creare un pool occorreva effettuare il deploy di un nuovo contratto.

Tutti i pool vengono tracciati dallo stato globale del PoolManager, entro il quale ogni pool possiede 4 variabili:

- **sqrtPriceX96**: radice quadrata del prezzo iniziale tra due token.
- **fee**: percentuale di fees.
- **tickspacing**: ogni quanti tick si può posizionare la liquidità.
- **hook**: indirizzo del contratto associato al pool.

Il gas viene risparmiato in parte anche grazie al fatto che gli eventuali hooks vengono richiamati soltanto dagli eventi a cui sono interessati, attraverso i permessi.

Funzioni elementari

Le funzioni principali analizzate in campo AMM e liquidity pools (swap, mint, burn...) sono disponibili anche in Uniswap v4, tuttavia come accennato, vengono chiamate attraverso l'interfaccia PoolManager, esempio di riutilizzo del codice.

Alcune differenze implementative possono essere osservate nella funzione di swap per esempio, dove vengono gestiti anche beforeSwap e afterSwap alle *righe 202 e 221* e dove vengono utilizzate le nuove variabili delta (*righe 198, 202, 206, 220, 221*):

```
198         BeforeSwapDelta beforeSwapDelta;
199         {
200             int256 amountToSwap;
201             uint24 lpFeeOverride;
202             (amountToSwap, beforeSwapDelta, lpFeeOverride) = key.hooks.beforeSwap(key, params, hookData);
203
204             // execute swap, account protocol fees, and emit swap event
205             // _swap is needed to avoid stack too deep error
206             swapDelta = _swap(
207                 pool,
208                 id,
209                 Pool.SwapParams({
210                     tickSpacing: key.tickSpacing,
211                     zeroForOne: params.zeroForOne,
212                     amountSpecified: amountToSwap,
213                     sqrtPriceLimitX96: params.sqrtPriceLimitX96,
214                     lpFeeOverride: lpFeeOverride
215                 }),
216                 params.zeroForOne ? key.currency0 : key.currency1 // input token
217             );
218         }
219
220         BalanceDelta hookDelta;
221         (swapDelta, hookDelta) = key.hooks.afterSwap(key, params, swapDelta, hookData, beforeSwapDelta);
```

Questi nuovi oggetti e variabili vengono utilizzati per gestire i cambiamenti di stato e le interazioni con i pool:

- BeforeSwapDelta **beforeSwapDelta**: dati per calcolare lo swap prima che avvenga.
- BalanceDelta **swapDelta**: rappresenta la differenza dei token coinvolti nello swap, secondo le regole dell'AMM.
- BalanceDelta **hookDelta**: il delta aggiuntivo causato dagli hook interessati a afterSwapReturnDelta.

Scenario d'uso completo

Per comprendere appieno le funzioni fondamentali su cui si basano gli AMM, è stato implementato e provato in testnet un market maker capace di compiere le operazioni principali del caso.

Creazione dei token

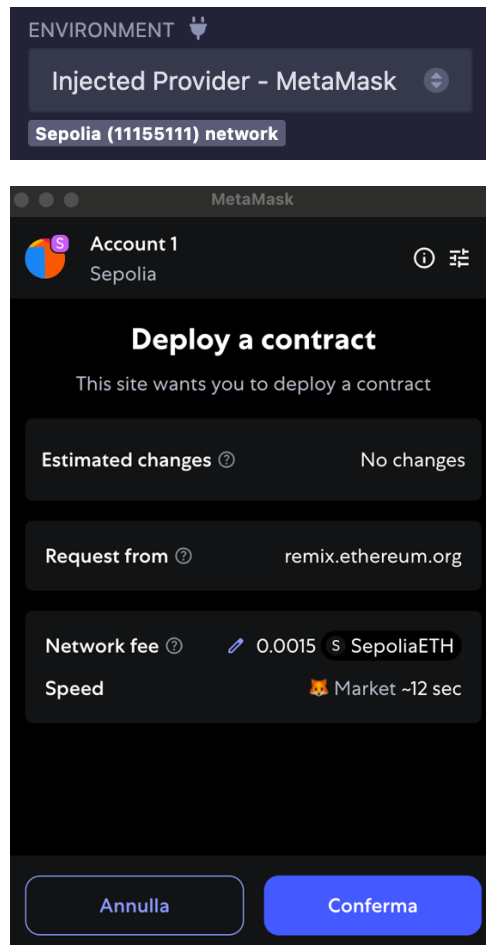
Per iniziare sono stati creati due contratti per definire i due token che il nostro AMM sarà capace di gestire. I token si chiamano **TokenA** e **TokenB**:

- Ereditano da **ERC20**.
- Chiamano il costruttore (*righe 8 e 25*).
- Introducono il modifier **onlyowner** per evitare che vengano manipolati da qualcuno che non è il proprietario (*righe 12 e 29*).
- Implementano la funzione **mint** per creare token e distribuirli a un address (*righe 17 e 34*).

```
5  contract TokenA is ERC20{
6      address owner;
7      error notTheOwner(address _address);
8      constructor() ERC20("TokenA", "A") {
9          owner = msg.sender;
10     }
11
12     modifier onlyOwner(){
13         if (msg.sender != owner) revert notTheOwner(msg.sender);
14         _;
15     }
16
17     function mint(address recipient, uint256 quantity) public onlyOwner{
18         _mint(recipient,quantity);
19     }
20 }
```

```
22  contract TokenB is ERC20{
23      address owner;
24      error notTheOwner(address _address);
25      constructor() ERC20("TokenB", "B") {
26          owner = msg.sender;
27     }
28
29     modifier onlyOwner(){
30         if (msg.sender != owner) revert notTheOwner(msg.sender);
31         _;
32     }
33
34     function mint(address recipient, uint256 quantity) public onlyOwner{
35         _mint(recipient,quantity);
36     }
37 }
```

È stato eseguito il deploy dei due contratti sulla testnet **Sepolia**, tramite **Remix IDE**, con un account su cui avevamo ottenuto dei **SepoliaETH** tramite dei faucets:



Automated Market Maker

In seguito è stato creato un contratto chiamato **bcsc2025amm**, che implementa un vero e proprio AMM che:

- Gestisce un **liquidity pool** su TokenA e TokenB.
- Permette ai liquidity providers di versare TokenA e TokenB in cambio di quote del pool con la **mint**.
- Permette ai liquidity providers di prelevare TokenA e TokenB in cambio di quote con la **burn**.
- Permette agli altri utenti di effettuare **swap** tra i due token.

Il contratto utilizza gli indirizzi dei due token creati in precedenza, che infatti verranno inseriti al momento del deploy e definisce le riserve di token e la liquidità mappata come quote di ciascun utente, come si può notare alle *righe 13 e 14*:

```
6      contract bcsc2025amm {
7          IERC20 public tokenA;
8          IERC20 public tokenB;
9
10         uint256 public reserveA;
11         uint256 public reserveB;
12
13         mapping(address => uint256) public liquidity;
14         uint256 public totalLiquidity;
15
16         constructor(address _tokenA, address _tokenB) {
17             tokenA = IERC20(_tokenA);
18             tokenB = IERC20(_tokenB);
19         }
```

La prima funzione è la **mint**, che permette di versare TokenA e TokenB in cambio delle **quote** di liquidity del pool:

```
21     function mint(uint256 amountA, uint256 amountB) external returns (uint256 shares) {
22         tokenA.transferFrom(msg.sender, address(this), amountA);
23         tokenB.transferFrom(msg.sender, address(this), amountB);
24
25         if (totalLiquidity == 0) {
26             shares = sqrt(amountA * amountB);
27         } else {
28             shares = min(
29                 (amountA * totalLiquidity) / reserveA,
30                 (amountB * totalLiquidity) / reserveB
31             );
32         }
33
34         require(shares > 0, "Liquidity mintata insufficiente");
35
36         liquidity[msg.sender] += shares;
37         totalLiquidity += shares;
38         reserveA += amountA;
39         reserveB += amountB;
40     }
```

In particolare la **mint**:

- Trasferisce i token all'indirizzo del contratto (*righe 22 e 23*).
- Calcola le quote da restituire in maniera diversa se è la prima fornitura di liquidità (*righe 25 e 26*).
- Si assicura che la liquidità minata sia sufficiente alla *riga 34*.
- Se è tutto apposto, incrementa la liquidità dell'utente e totale del pool e aggiorna i valori delle riserve di token.

La **burn** invece consente l'operazione opposta, restituendo TokenA e TokenB in cambio delle quote di liquidità (come dai calcoli alle *righe 45 e 46*):

```
42     function burn(uint256 shares) external returns (uint256 amountA, uint256 amountB) {
43         require(liquidity[msg.sender] >= shares, "Non hai abbastanza quote");
44
45         amountA = (shares * reserveA) / totalLiquidity;
46         amountB = (shares * reserveB) / totalLiquidity;
47
48         liquidity[msg.sender] -= shares;
49         totalLiquidity -= shares;
50         reserveA -= amountA;
51         reserveB -= amountB;
52
53         tokenA.transfer(msg.sender, amountA);
54         tokenB.transfer(msg.sender, amountB);
55     }
```

In particolare:

- Si assicura che l'utente abbia le quote che vuole “bruciare”.
- Calcola le quantità di token in base alla liquidità totale del pool.
- Aggiorna le liquidità e le riserve.
- Trasferisce i token all'indirizzo dell'utente.

L'ultima funzione è la **swap**, che permette di scambiare TokenA e TokenB, eseguendo le seguenti operazioni:

- Controlla che il token che si sta cercando di scambiare sia TokenA o TokenB alla *riga 59*.
- Trasferisce i token all'indirizzo del contratto (*riga 64*).
- Calcola l'output aggiungendo una fee dello 0.3% (*righe 66 e 67*).
- Se l'output calcolato non è nullo, trasferisce i token risultanti all'utente alla *riga 71*.
- Aggiorna le riserve.

```

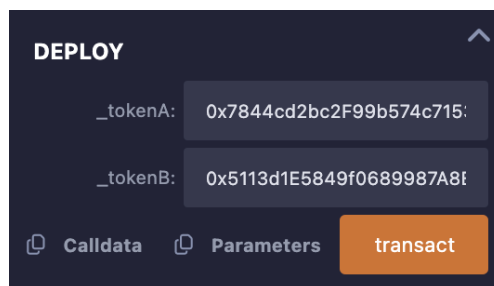
57     function swap(address tokenIn, uint256 amountIn) external returns (uint256 amountOut) {
58         bool isAtoB = tokenIn == address(tokenA);
59         require(isAtoB || tokenIn == address(tokenB), "INVALID_TOKEN");
60
61         (IERC20 tokenFrom, IERC20 tokenTo, uint256 reserveFrom, uint256 reserveTo) =
62             isAtoB ? (tokenA, tokenB, reserveA, reserveB) : (tokenB, tokenA, reserveB, reserveA);
63
64         tokenFrom.transferFrom(msg.sender, address(this), amountIn);
65
66         uint256 amountInWithFee = (amountIn * 997) / 1000;
67         amountOut = (amountInWithFee * reserveTo) / (reserveFrom + amountInWithFee);
68
69         require(amountOut > 0, "INSUFFICIENT_OUTPUT");
70
71         tokenTo.transfer(msg.sender, amountOut);
72
73         if (isAtoB) {
74             reserveA += amountIn;
75             reserveB -= amountOut;
76         } else {
77             reserveB += amountIn;
78             reserveA -= amountOut;
79         }
80     }

```

Il modello è **constant product** e segue la formula:

$$\frac{amountInWithFee \cdot reserveB}{reserveA + amountInWithFee}$$

Anche di questo contratto è stato eseguito il deploy, inserendo come indirizzi dei token quelli ottenuti con i deploy dei rispettivi contratti:



DEPLOY

_tokenA: 0x7844cd2bc2F99b574c715:

_tokenB: 0x5113d1E5849f0689987A8f

Calldata Parameters transact

Test

Per prima cosa abbiamo mintato **13 TokenA** e **10 TokenB** nel nostro account (formato decimale a 18):

MINT

recipient: 0x515Ffd65894B3a3569468

quantity: 13000000000000000000

Calldata Parameters transact

balanceOf 0x515Ffd65894B3a3569468

0: uint256: 13000000000000000000

In seguito abbiamo chiamato la **approve** con l'indirizzo del contratto di AMM, per permettergli di gestire per conto nostro fino a 200 TokenA e 200 TokenB:

APPROVE

spender: 0xc5110d8c44a6a6d7b74e58

value: 20000000000000000000

Calldata Parameters transact

totalLiquidity

0: uint256: 0

In questo momento la **liquidità** del pool è nulla, infatti non è stato versato ancora nessun token:

totalLiquidity

0: uint256: 0

Come prossimo passo abbiamo versato 10 TokenA e 10 TokenB tramite la **mint**:

The image shows two screenshots from a web application. The top screenshot is titled 'MINT' and features two input fields: 'amountA:' and 'amountB:', both containing the value '1000000000000000000'. Below these fields are three buttons: 'Calldata', 'Parameters', and 'transact' (which is highlighted in orange). The bottom screenshot is titled 'Transaction request' and displays 'Estimated changes'. It shows 'You send' with two entries: '- 10' next to the address '0x7844c...a0512' and '- 10' next to the address '0x5113d...740A7'.

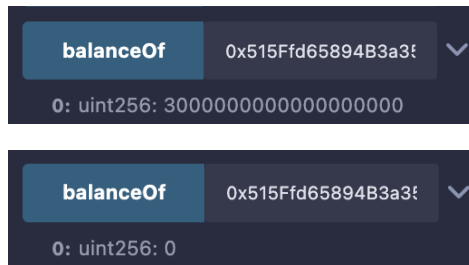
In cambio di questi token, abbiamo ricevuto **10 quote** del pool (LP token), che ora risultano assegnate al nostro address, e che rappresentano anche la liquidità totale.

Come si può vedere dal codice, le quote del primo mint vengono calcolate come la radice quadrata del prodotto delle quantità dei due token.

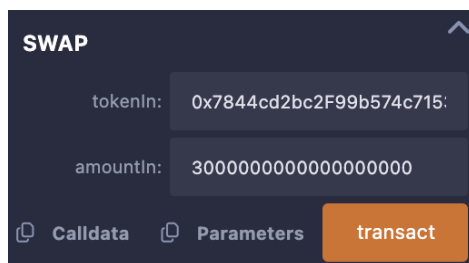
The image shows a screenshot of a 'liquidity' pool interface. At the top, there is a header 'liquidity' followed by the address '0x515Ffd65894B3a3f' and a dropdown arrow. Below this, the text '0: uint256: 1000000000000000000' is displayed. A list of variables is shown in a vertical stack of buttons: 'reserveA', 'reserveB', 'tokenA', 'tokenB', and 'totalLiquidity'. At the bottom, the text '0: uint256: 1000000000000000000' is displayed again.

Ora possiamo eseguire la **swap**, per esempio dei 3 TokenA che ci rimanevano.

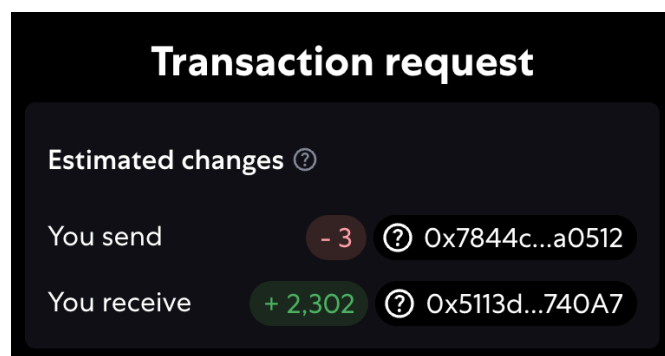
Prima dello swap abbiamo 3 TokenA e 0 TokenB:



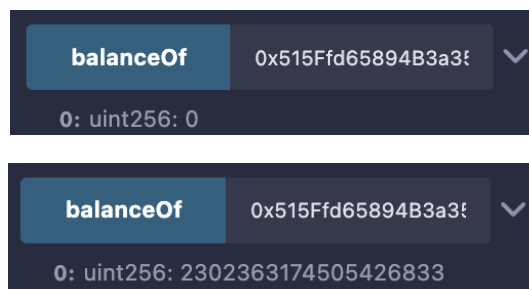
Per eseguire lo swap dobbiamo specificare anche l'address del contratto di TokenA:



In cambio dei **3 TokenA** otteniamo **2.302 TokenB**, calcolati con la formula descritta al paragrafo precedente:

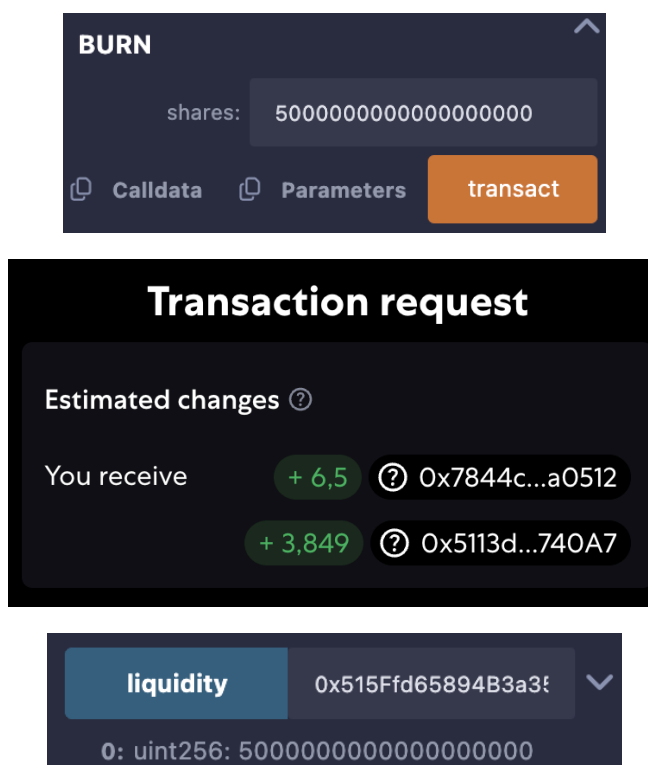


Possiamo vedere che ora il nostro balance di TokenA è vuoto, mentre quello di TokenB contiene i TokenB appena ricevuti:



È possibile anche notare che le **riserve** del pool ora contengono 13 TokenA e circa 7.6 TokenB, che in totale sono più dei 10+10 token iniziali, di conseguenza ora le quote hanno un valore superiore e se non fossimo stati noi a fare la swap, prelevando i fondi avremmo guadagnato una plusvalenza.

Per provare a prelevare liquidità, eseguiamo la **burn** con 5 delle nostre 10 quote e osserviamo come la liquidità diminuisce e i nostri bilanci dei token aumentano ancora una volta:



Possiamo notare che per 5 quote abbiamo ottenuto 6.5 TokenA e 3.849 TokenB, mentre all'inizio 5 quote valevano leggermente meno: 5 TokenA e 5 TokenB (a parità di valore dei due token).

Vulnerabilità

Una problematica particolare riscontrata nella fase di test è quella relativa all'**underflow** dello swap.

Le funzioni prendono in input parametri in formato decimale a 18, quindi 1 token si scrive 1000000000000000000.

Se non si tiene conto di questo dettaglio e si effettuano gli esperimenti precedenti utilizzando quantità infinitesimali di token, scrivendo semplicemente 1 (1×10^{-18} token), quando si prova a effettuare uno swap, la quantità di token ricevuta in cambio non rispetterà la formula descritta in precedenza.

In particolare, si era provato a versare con una mint nel pool 10×10^{-18} **TokenA** e 10×10^{-18} **TokenB**, per poi eseguire lo swap di 3×10^{-18} **TokenA**.

La quantità di TokenB restituita era pari a 1×10^{-18} invece dei $2,3 \times 10^{-18}$ che ci si aspettava, a causa della formula della funzione di swap che tronca i numeri molto piccoli.

Conclusioni

Con i contratti precedentemente illustrati è possibile implementare il cuore di un Automated Market Maker basato su un pool tra TokenA e TokenB.

Ricollegandosi alla teoria, tramite questo nucleo è possibile osservare molte dinamiche che avvengono continuamente in ambito DeFi basata su AMM e Liquidity Pools. In particolare:

- I **liquidity providers** versano **token** in cambio di **quote** del pool.
- Qualunque utente può eseguire lo **swap** tra token, sulla base della formula dell'AMM.
- I liquidity providers possono **bruciare** le quote e prelevare i token.
- Il pool aumenta il valore attraverso le **fees** e i liquidity providers possono trarre guadagno da questo processo, dato che le quote valgono sempre di più se tutto va per il verso giusto.

Tutto il materiale è presente nel repository <https://github.com/Matti02co/BCSC2025-CoccoLepuri>.

Bibliografia

Andrea Cesaretti - Auto-Paired DeFi Pools: Illusory Liquidity and Price Manipulation. A Critical Review.

Yongge Wang UNC Charlotte - Automated Market Makers for Decentralized Finance (DeFi).

Hayden Adams, Noah Zinsmeister, Dan Robinson – Uniswap v2 Core.

<https://www.coinbase.com/it>

<https://github.com/Uniswap/v2-core/tree/master>

<https://docs.uniswap.org/contracts/v2/overview>

<https://docs.balancer.fi/>

<https://resources.curve.finance/>

<https://www.horizen.io/academy/liquidity-in-defi/>

<https://www.ulam.io/blog/best-crypto-exchange-liquidity-providers#>

<https://www.alloy.capital/articles/institutional-defi-the-asset-management-service-provider-landscape>

<https://www.fireblocks.com/>

<https://docs.beefy.finance/>

<https://coinmarketcap.com/academy/article/a-deep-dive-into-how-the-top-daos-work>

<https://immunefi.com/bug-bounty-program/>

<https://chain.link/whitepaper>

<https://tokenterminal.com/explorer/projects/uniswap/financial-statement>

<https://coinmarketcap.com/currencies/curve-dao-token/>

<https://github.com/uniswapfoundation/v4-template/blob/main/src/Counter.sol>

<https://github.com/Uniswap/v4-core/blob/main/src/PoolManager.sol>

Licenza

© 2025 Mattia Cocco & Tomas Lepuri – Licenza CC BY 4.0